JUMP GAME

Company: Microsoft
Round: Online Assessment
Year: 2022
Difficulty: Easy - Medium
Topic: Arrays, BFS, Queue, Visited Array
Source: https://algo.monster/problems/jump_game

Problem Description:

We are given an array of non-negative integers called arr and a starting position called start. The value at each index tells us how far we have to jump from that index.

For an index i, we have two choices:

- Move left by arr[i] positions.
- Move right by arr[i] positions.

Our goal is simple:

Can we reach any index whose value is **0**?
If we can reach 0, return true.
If there is no possible way to reach 0, return false.

Important point
We must stay inside the array. If a jump takes us outside the array, we simply cannot make that jump.

Input Format:

The input contains:

- An integer array arr
- An integer start, which represents our starting index

Output Format:

Return : true,
if we can reach an index containing 0.
Otherwise, Return : false

Sample Input:

arr = [3, 4, 2, 3, 0, 3, 1, 2, 1]
start = 7

Sample Output:

true

Explanation:

Let's understand the first example step by step.

arr = [3, 4, 2, 3, 0, 3, 1, 2, 1]

The indexes are:

Index: 0 1 2 3 4 5 6 7 8

Value: 3 4 2 3 0 3 1 2 1

We start at:

start = 7

At index 7:

arr[7] = 2

This means we can move exactly 2 positions.

So we have two choices:

7 - 2 = 5

or

7 + 2 = 9

Index 9 does not exist because the last index is 8.

So we can only move to index 5.

Now we are at index 5:

arr[5] = 3

Again, we can move 3 positions:

5 - 3 = 2

or

5 + 3 = 8

We can explore both possibilities.

From index 2:

arr[2] = 2

So we can move to:

2 - 2 = 0

or

2 + 2 = 4

Index 4 is especially important because:

arr[4] = 0

We have reached 0.

Therefore, the answer is:

true

One valid path is:

7 → 5 → 2 → 4

What Makes This Problem Slightly Tricky?

At every index, we have two possible directions**.**

For example, from index 5, we can go to:

2

or:

8

So there can be many different paths.

Also, we might come back to an index that we have already visited.

For example:

A → B → C → A → B → C → ...

This could continue forever.

Therefore, we need a way to remember:

"Have I already visited this index?"

This is where the visited array comes in.

What is a Visited Array?

A visited array is simply an array of boolean values.

For example:

visited = [false, false, false, false, false]

Initially, every position is false, meaning we haven't visited anything.

When we visit index 2, we change:

visited[2] = true

Now we know that index 2 has already been checked.

If we encounter index 2 again later, we don't need to check it again.

This prevents unnecessary work and also prevents infinite loops.

Approach to Solve This Problem:

We can think of every index as a place we can visit.

From one index, we can move to at most two other indexes:

left
↑
current
↓
right

We need to explore these possible positions until we either find 0 or run out of positions.

For this, we use **BFS (Breadth-First Search)**.

Don't worry if BFS sounds complicated. The basic idea here is very simple.

Step 1: Start from the given index

We start from:

start

We put this index into a **queue**.

A queue follows:

First In, First Out (FIFO)

Just like people standing in a line — the person who comes first is served first.

Step 2: Take one index from the queue

Suppose the queue contains:

[7]

We take 7 and check it.

If:

arr[7] == 0

we are done and return true.

Otherwise, we calculate where we can jump.

Step 3: Find the left and right positions

If we are currently at index i, then:

left = i - arr[i]
right = i + arr[i]

For example, if:

i = 5
arr[5] = 3

then:

left = 5 - 3 = 2
right = 5 + 3 = 8

So we can explore indexes 2 and 8.

Step 4: Check whether the positions are valid

A position is valid only if it is inside the array.

For an array of length n:

0 <= index < n

So if a jump gives us -1, we cannot use it.

Similarly, if a jump gives us 10 but the last index is 8, we cannot use it.

Step 5: Check whether we have already visited it

Before adding an index to the queue, we check:

!visited[index]

The ! means **"not"**.

So:

!visited[index]

means:

"This index has not been visited yet."

If it has not been visited, we mark it:

visited[index] = true;

and add it to the queue.

Step 6: Continue until we find zero

We keep taking indexes from the queue and exploring their left and right jumps.

There are only two possible outcomes:

We find 0
Return:
true

We explore everything but never find 0

Return : false

Java Code:

```java
import java.util.*;

public class Main {

    public static boolean canReach(int[] arr, int start) {

        boolean[] visited = new boolean[arr.length];

        Queue<Integer> queue = new LinkedList<>();

        // Start from the given index
        queue.add(start);
        visited[start] = true;

        while (!queue.isEmpty()) {

            int index = queue.remove();

            // If we reach a value of 0
            if (arr[index] == 0) {
                return true;
            }

            // Calculate the two possible jumps
            int left = index - arr[index];
            int right = index + arr[index];

            // Move left
            if (left >= 0 && !visited[left]) {
                visited[left] = true;
                queue.add(left);
            }

            // Move right
            if (right < arr.length && !visited[right]) {
                visited[right] = true;
                queue.add(right);
            }
        }

        // No path can reach 0
        return false;
    }

    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Enter number of elements
        int n = sc.nextInt();

        int[] arr = new int[n];

        // Enter array elements
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // Enter starting index
        int start = sc.nextInt();

        boolean result = canReach(arr, start);

        System.out.println(result);

        sc.close();
    }
}
```

Solution:

The code follows three simple steps:

1. Start from the given index and add it to a queue.
2. Explore both possible jumps, left and right, while using a visited array so that we don't check the same index again.
3. Return true if we reach a value of 0. If every possible position is checked without finding 0, return false.

Understanding the Important Part of the Code:

The most important part is:

int left = index - arr[index];

int right = index + arr[index];

This calculates the two positions we can jump to.

Then we check:

if (left >= 0 && !visited[left])

This means:

"Is the left position inside the array, and have we not visited it before?"

Similarly:

if (right < arr.length && !visited[right])

means:

"Is the right position inside the array, and have we not visited it before?"

Finally:

if (arr[index] == 0)

checks whether we have reached our target.

Complexity Analysis

Time Complexity: O(n)

There are n indexes in the array.
Because of the visited array, every index is processed at most once**.**
At each index, we check only two possible moves: left and right.
Therefore, the time complexity is:
O(n)

Space Complexity: O(n)

We use:

- A visited array of size n
- A queue that can contain up to n indexes

Therefore: O(n)space is required.